

SOFTWARE DESIGN & ARCHITECTURE

Assignment 2: Architectural Styles

Student Name:	Muhammad Aun Anis
Roll Number:	24P-3017
Course:	SE – Software Design & Architecture
Assignment:	Assignment #2
Topic:	Software Architectural Styles
Total Marks:	50

Scenario 1 — Smart Hospital Management System

Question 9: Architectural Style for the AI-Based Diagnostic Module

Chosen Architecture: Blackboard Architecture

The AI-based diagnostic module is best suited to the **Blackboard architectural style**. The Blackboard pattern is designed for problems where no single deterministic algorithm can produce a solution, and multiple independent expert modules (knowledge sources) must each contribute partial solutions to a shared data structure (the blackboard).

Justification with reference to Blackboard concepts:

- **Blackboard (shared data store):** The central blackboard holds the patient's current diagnostic state — symptoms entered at reception, vitals from monitoring devices, lab results, and imaging data. All knowledge sources read from and write to this shared space.
- **Knowledge Sources (KS):** Each independent medical module acts as a KS. For example: a Symptom Analyser KS, a Lab Result Interpreter KS, an Imaging Pattern Recogniser KS, and a Vitals Evaluator KS. None of these can independently produce a diagnosis — they each contribute ranked hypotheses to the blackboard.
- **Control Component:** A controller (scheduler) monitors the blackboard and activates the appropriate KS when new data arrives. For instance, when lab results are posted, the Lab Result Interpreter KS is triggered automatically, satisfying the requirement that physicians are alerted as soon as results are ready.
- **Opportunistic Problem-Solving:** The system does not follow a fixed sequence. KSs fire when their triggering conditions are met, allowing the diagnosis to evolve incrementally as more data becomes available — ideal for a clinical environment where data arrives asynchronously.

Question 10: Architectural Style for Module Separation

Chosen Architecture: Layered (N-Tier) Architecture

The separation of Reception, Doctor, Lab, Pharmacy, and Administration modules is best achieved through **Layered Architecture**. In a layered system, each module is assigned to a specific layer and is only permitted to communicate with adjacent layers, enforcing strict access boundaries.

Justification with reference to Layered Architecture concepts:

- **Separation of Concerns:** Each department (Reception, Doctor, Lab, Pharmacy, Admin) is encapsulated within its own layer or sub-layer. A receptionist's UI layer calls only the patient registration service layer, and cannot reach the clinical notes layer that belongs to Doctors.
- **Role-Based Access Enforcement:** The layered model naturally enforces role-based security — requests from Reception pass only through the registration and billing service layers, while Doctors access the clinical and prescription layers. This maps directly to the requirement that receptionists cannot access clinical notes.
- **Maintainability:** Changes to the Pharmacy module (e.g., updating the drug dispensing rules) do not affect the Lab or Doctor modules, since each layer has a well-defined interface.

Disadvantages:

1. **Performance Overhead from Layering:** In a strict layered architecture, every request must pass through all intermediate layers even when lower layers are not needed. In a time-critical hospital scenario (e.g., an emergency physician needing real-time lab results), this multi-layer traversal can introduce latency.

2. **Complexity of Blackboard Coordination:** Managing multiple KSs with an asynchronous controller introduces significant development and debugging complexity. Ensuring consistency of the shared blackboard when multiple KSs write simultaneously (e.g., lab results and vitals arriving at the same moment) requires careful concurrency control, which is non-trivial in a production hospital system.

Question 13: Alternative Architecture and Comparison

Alternative: Rule-Based Expert System (Pipe and Filter)

A **Pipe and Filter** architecture could be proposed as an alternative for the diagnostic module, where patient data flows sequentially through filters: Symptom Filter → Lab Filter → Imaging Filter → Vitals Filter → Diagnosis Output. Each filter transforms the data and passes it to the next.

Why Blackboard is Better:

Pipe and Filter imposes a **fixed sequential order** of processing, which is unsuitable for clinical diagnosis. In reality, imaging data may arrive before lab results, or a patient may have no imaging data at all. A Pipe and Filter system would either stall waiting for all inputs or require complex branching logic to handle missing stages. The Blackboard architecture, by contrast, allows KSs to fire opportunistically whenever their required data is available, making it far more robust to the asynchronous, incomplete, and multi-source nature of clinical data. Furthermore, Blackboard explicitly supports the scenario requirement that 'no single algorithm can solve the diagnosis problem' — multiple independent KSs each contribute partial knowledge, which is the core design philosophy of Blackboard.

Scenario 2 — Global E-Commerce Platform

Question 14: Overall Architectural Style

Chosen Architecture: Microservices Architecture

The most suitable overall architectural style is **Microservices Architecture**. This platform must support seven independently deployable modules, heterogeneous technology stacks, selective scaling, and data isolation for sensitive services — all hallmarks of a microservices system.

Justification with reference to Microservices concepts:

- **Single Responsibility per Service:** Each module (User Accounts, Product Catalog, Order Management, Payment Processing, Inventory, Notifications, Recommendation Engine) is implemented as an independent service with its own codebase, deployment pipeline, and runtime. This directly supports the requirement that each team of 5–8 developers deploys their module independently.
- **Decentralised Data Management:** Microservices mandates that each service owns its database. The Payment module's requirement for a separate secure database is a natural fit — in microservices, no other service can directly query the Payment database, enforcing data isolation by design.
- **Technology Heterogeneity:** Microservices places no constraint on implementation technology. The Recommendation Engine can use Python/ML frameworks (e.g., TensorFlow, FastAPI) while other services use Java (Spring Boot) or Node.js, communicating through technology-agnostic APIs (REST or message queues).
- **Independent Scalability:** During peak sales events, only the Orders and Payments services need to be scaled horizontally (e.g., via Kubernetes), while User Accounts or Notifications remain at baseline capacity — impossible with a monolithic architecture.

Question 15: Decoupling the Notification Module from the Order Module

Architectural Style / Pattern Used: Event-Driven Architecture (Message Broker Pattern)

The Notification module is decoupled from the Order module using an **Event-Driven Architecture** with a **Message Broker** (e.g., Apache Kafka or RabbitMQ). The Order service publishes domain events to a topic/queue; the Notification service subscribes to those events and reacts independently.

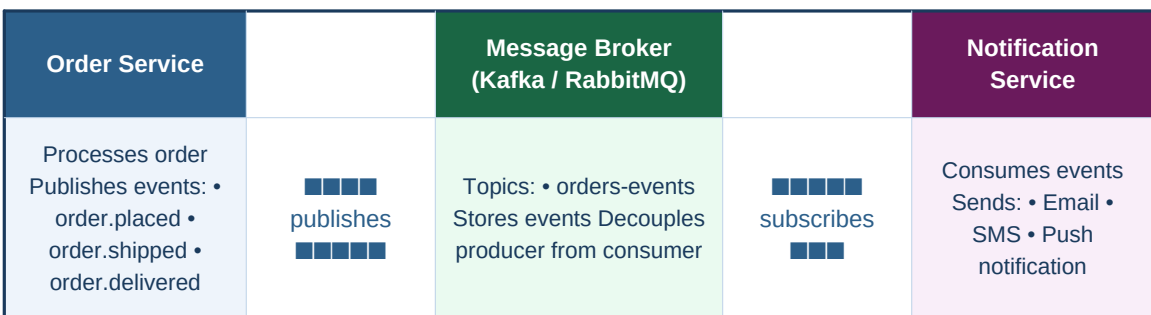


Figure 2: Event-Driven decoupling of Order Service and Notification Service via Message Broker

The Order module has **no direct dependency** on the Notification module. It simply fires an event and continues processing. The broker guarantees delivery; the Notification service handles email, SMS, and push independently. If the Notification service is temporarily down,

events queue in the broker and are processed upon recovery — the Order service is unaffected.

Question 16: How Microservices Addresses Key Challenges

All three challenges are addressed by Microservices, but the most **distinctive** challenge resolved here is **Independent Scaling**, as it is a core driver for choosing Microservices over alternatives like SOA.

Technology Diversity:

Each microservice is a self-contained deployment unit with its own technology stack. The Recommendation Engine team selects Python + scikit-learn/TensorFlow independently of the Java Spring Boot teams. Communication via REST APIs or gRPC means the internal implementation language is irrelevant to other services.

Independent Scaling:

During a flash sale, the Orders and Payments microservices can be scaled to 50+ instances via a container orchestrator (Kubernetes) while the User Accounts and Inventory services remain at 2–3 instances. This is possible because each service has its own container image and resource allocation, not shared with others.

Data Isolation:

The Payment service owns a dedicated encrypted database (e.g., PostgreSQL with row-level encryption). No other service has credentials to this database. All inter-service communication goes through the Payment service's API, which enforces authentication and authorisation before exposing any financial data.

Question 17: Advantages and Disadvantages of Microservices for this Scenario

Advantages:

1. **Independent Deployment and Team Autonomy:** Each team of 5–8 developers can release their service (e.g., Product Catalog) on its own sprint cycle without coordinating a monolithic release. A bug fix in Inventory does not require redeploying Orders or Payments. This directly satisfies the core platform requirement and enables the startup to scale its engineering organisation as it grows to 100 million users.
2. **Fault Isolation:** If the Recommendation Engine crashes (e.g., an ML model consumes too much memory), it does not bring down Order Management or Payments. With proper circuit breakers (e.g., Resilience4j), the platform degrades gracefully — users can still browse and purchase, just without personalised recommendations. This is critical for a platform targeting 100 million users where total downtime is unacceptable.

Disadvantages:

1. **Distributed System Complexity:** Operations that span multiple services (e.g., placing an order involves Orders, Inventory, and Payments) cannot use simple database transactions. Implementing distributed sagas or two-phase commits for order consistency is significantly more complex than a single ACID transaction in a monolith, increasing the risk of partial failures and data inconsistencies.
2. **Operational Overhead:** Running seven or more independent services requires a mature DevOps infrastructure: container orchestration (Kubernetes), service discovery, distributed tracing, centralised logging, and API gateways. For a startup at launch with limited DevOps

experience, this operational burden may slow initial development and increase infrastructure costs before the scale benefits are realised.

Question 18: Microservices vs SOA — Which is Better and Why?

Dimension	Microservices	SOA
Service Granularity	Fine-grained; one service per business capability (e.g. Orders, Payments)	Coarse-grained; services map to broader business functions (e.g. Order Service)
Communication	Lightweight REST / message queues (Kafka)	Heavyweight ESB (Enterprise Service Bus); SOAP/WSDL
Data Management	Each service owns its database (enforced isolation)	Services often share a common enterprise database
Deployment	Independent per-service deployment (containers)	Typically deployed together or via ESB updates
Technology Choice	Polyglot — each service chooses its own stack (Python, Java, Node.js)	Strictly constrained to enterprise-approved technologies
Scalability	Individual service scaling (Orders scales independently of Payments)	Scaling is typically at the ESB or application server level
Fault Isolation	Failure in one service does not cascade (circuit breakers)	ESB is a single point of failure

Verdict: Microservices is the better choice for this platform. SOA's reliance on an Enterprise Service Bus (ESB) introduces a central bottleneck that undermines independent deployment and creates a single point of failure — unacceptable for a platform targeting 100 million users. SOA's coarse service granularity would also force the Payment team to share deployment cycles with other services, violating the requirement of independent releases. Microservices, with lightweight communication (REST/Kafka), per-service databases, and container-based deployment, is architecturally aligned with every stated requirement of this e-commerce platform.

Scenario 3 — National Payroll Processing System

Question 19: Most Suitable Architectural Style

Chosen Architecture: Batch Sequential Architecture

The **Batch Sequential** architectural style is the most appropriate choice for this payroll system. In Batch Sequential, the system processes a large dataset in discrete, ordered stages. Each stage completes fully before the next begins, and data passes from one stage to the next as a complete batch.

Justification:

- **No real-time requirement:** The scenario explicitly states that real-time processing is NOT required. All data is collected by the 25th and processed in a single end-of-month run — a perfect fit for batch processing.
- **Fixed Sequential Steps:** The six processing stages (collect → validate → apply rules → generate payslips → transfer to bank → archive) must execute in a strict, defined order. Batch Sequential enforces this sequencing by design.
- **Abort-on-Failure Semantics:** The requirement that a failure in any step aborts and restarts the entire run aligns with Batch Sequential's all-or-nothing processing model, where intermediate partial results are not propagated forward.
- **Large Volume, Infrequent Processing:** Processing 500,000 payroll records once per month is the classic use case for batch architecture — high throughput over a defined time window, not low-latency streaming.

Question 20: Architectural Flow Diagram

BATCH SEQUENTIAL PAYROLL PROCESSING FLOW	
Stage 1: Data Collection	Attendance & leave records collected from all departments by 25th of month
▼ (next stage only starts after this stage completes successfully)	
Stage 2: Data Validation & Cleaning	Remove duplicates, correct errors, check completeness of records
▼ (next stage only starts after this stage completes successfully)	
Stage 3: Payroll Calculation	Apply tax deductions, allowances, and bonuses per government rules
▼ (next stage only starts after this stage completes successfully)	
Stage 4: Payslip Generation	Generate PDF payslip for each of 500,000 civil servants
▼ (next stage only starts after this stage completes successfully)	
Stage 5: Bank Transfer	Transfer final salary amounts to the national banking system

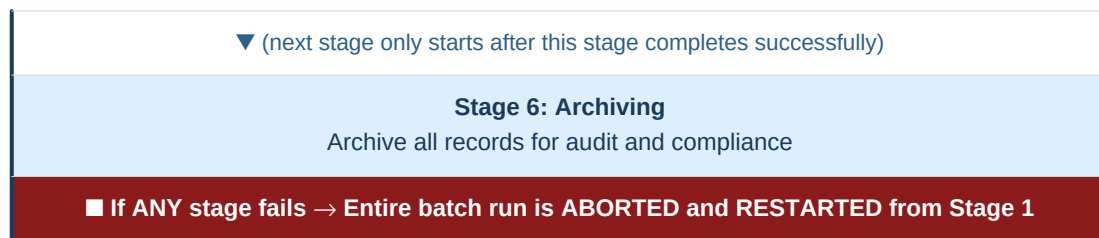


Figure 3: Batch Sequential Architectural Flow for National Payroll Processing System

Question 21: Effect of Stage Failure

In a Batch Sequential architecture, **a failure in any stage halts the entire run**, and the whole process must be restarted from Stage 1. For example, if Stage 3 (Payroll Calculation) encounters a corrupt rule file and crashes, Stages 4, 5, and 6 never execute, and no payslips are generated or bank transfers made.

Why this is acceptable in this context:

Given that payroll is processed once per month with a known schedule and a several-day buffer before month-end, a failed run can be diagnosed, corrected, and restarted without missing the salary payment deadline. The abort-on-failure behaviour is actually **desirable here** — it prevents incorrect partial payments (e.g., some employees receiving salaries calculated without correct tax rules) which would be far more damaging than a delayed restart. Government payroll demands correctness over speed.

Why this could be problematic:

If a failure occurs very close to the payment deadline (e.g., Stage 5 bank transfer fails on the 31st), restarting from Stage 1 means reprocessing all 500,000 records again, which may take many hours and cause salary delays. There is also no partial recovery — even if only 100 records caused the Stage 3 failure, all 500,000 must be reprocessed.

Question 22: Scenario Where This Architecture Would Be Completely Inappropriate

Inappropriate Scenario: Real-Time Stock Trading Platform

A stock trading platform (e.g., processing buy/sell orders for equities) would be completely incompatible with Batch Sequential architecture.

Why it is inappropriate:

- **Real-time requirement:** Stock trades must be executed and confirmed in milliseconds. Batch Sequential processes data in bulk at fixed intervals — a trading order queued for end-of-day batch processing would be economically catastrophic as prices move within microseconds.
- **Continuous data streams:** Stock markets generate a continuous, unbounded stream of price ticks and order events. Batch Sequential requires a defined, finite dataset to process — it has no mechanism to handle incoming events during a batch run.
- **Abort-on-failure unacceptable:** A failed calculation in one trade must not abort all other active trades. Fault isolation at the individual transaction level is mandatory — the diametric opposite of Batch Sequential's all-or-nothing model.

In this scenario, an **Event-Driven or Stream Processing architecture** (e.g., Apache Kafka Streams) would be required.

Section 4: Student Self-Evaluation Checklist

The following checklist confirms that all assignment requirements have been fulfilled prior to submission.

Checklist Item	Yes	No
I have answered all three scenarios completely.	☒	☐
Each answer includes an architectural diagram with labeled components.	☒	☐
I have justified my architecture choice with specific reference to the scenario.	☒	☐
I have discussed at least two advantages and two disadvantages per scenario.	☒	☐
I have proposed and compared an alternative architecture (Scenarios 1 & 2).	☒	☐
My diagrams are clear, legible, and directly relevant to the scenario.	☒	☐
My Name, Student ID, and Section are clearly written on the cover page.	☒	☐
I have reviewed my work for spelling and clarity before submission.	☒	☐
I have not copied from any classmate or online source.	☒	☐

Student Name: Muhammad Aun Anis
Roll Number: 24P-3017
Course: SE – Software Design & Architecture